JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 11 OF 17 - STUDY STEP DSA-11

Stacks, Queues, Deques, and Monotonic Patterns

LIFO, FIFO, Sliding Extrema, Next-Greater State, and Java ArrayDeque

BY

Vinay Reddy Kalluri

Choose LIFO, FIFO, deque, priority, or monotonic state correctly and implement the pattern with safe Java collection APIs.

STACK | QUEUE | DEQUE | MONOTONIC | WINDOWS | ARRAYDEQUE

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `DSA 10 - Linked Lists`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume unifies ordering data structures around the state each one preserves, then applies monotonic structures to linear-time interview patterns.

Readiness map

Decision	Evidence
START HERE	The most recent unresolved item matters; Work must be processed in arrival order; Both ends need efficient updates; Dominated candidates can be discarded while preserving extrema.
PREPARE FIRST	JAVA-01 and DSA-01 through DSA-10; Array and linked-list fundamentals.
MOVE ON WHEN	Use ArrayDeque for stack and queue roles; Distinguish queue, deque, and priority semantics; Implement next-greater and sliding-window maximum; Explain amortized linear monotonic processing.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 10 - Linked Lists	**YOU ARE HERE** **DSA 11 - Stacks, Queues, and Deques**	DSA 12 - Binary Search

A note from Vinay

A stack, queue, or deque earns its place by making an ordering invariant cheap. Say which candidate is removed next and why stale candidates can never become useful again.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Stack, Queue, and Deque Foundations Before Monotonic Patterns

These structures are not distinguished by what they store. They are distinguished by which stored item is accessible next.

Three access policies

TEXT

```
Stack: last in, first out       push -> [newest ... oldest] -> pop
Queue: first in, first out      add  -> [oldest ... newest] -> remove
Deque: both ends are available  front <-> [items] <-> back
```

A stack fits nested work and most-recent unresolved state. A queue fits arrival order and breadth-first layers. A deque fits work at both boundaries and can implement either policy.

Use ArrayDeque deliberately

For new interview code, prefer `ArrayDeque` over the legacy `Stack` class.

Role	Insert	Inspect	Remove
Stack at front	`push(x)`	`peek()`	`pop()`
Queue	`addLast(x)`	`peekFirst()`	`removeFirst()`
Deque front	`addFirst(x)`	`peekFirst()`	`removeFirst()`
Deque back	`addLast(x)`	`peekLast()`	`removeLast()`

The throwing forms (`removeFirst`, `pop`) fail on empty input. The nullable forms (`pollFirst`, `peek`) return null. `ArrayDeque` does not permit null elements, which keeps null available as an empty-result signal.

JAVA

```java
Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.push(20);
System.out.println(stack.pop()); // 20

Deque<Integer> queue = new ArrayDeque<>();
queue.addLast(10);
queue.addLast(20);
System.out.println(queue.removeFirst()); // 10
```

Do not mix ends without naming the policy. `addLast` followed by `removeLast` is a stack; `addLast` followed by `removeFirst` is a queue.

WHY IT WORKS | Stack invariant: unresolved nested state

For delimiter validation, the stack holds opening delimiters that have not yet found a matching close. The top must match the next closing delimiter.

JAVA

```java
static boolean balanced(String text) {
    Deque<Character> openings = new ArrayDeque<>();
    for (char token : text.toCharArray()) {
        if (token == '(' || token == '[' || token == '{') {
            openings.push(token);
        } else if (token == ')' || token == ']' || token == '}') {
            if (openings.isEmpty() || !matches(openings.pop(), token)) {
                return false;
            }
        }
    }
    return openings.isEmpty();
}
```

The final emptiness check matters: `"(("` never produces a mismatched closer, but remains incomplete.

WHY IT WORKS | Queue invariant: discovered but not processed

In breadth-first search, the queue contains discovered nodes whose outgoing relationships have not yet been processed. Mark a node visited when enqueuing, not when dequeuing, to prevent multiple parents from adding the same node repeatedly.

Circular queue mechanics

An array-backed bounded queue needs:

- `head`: index of the next element to remove;
- `size`: current number of elements;
- capacity: array length;
- insertion index: `(head + size) % capacity`.

Tracking size separates empty from full without sacrificing one slot. Guard capacity zero before modulo.

From ordinary stack to monotonic stack

A monotonic stack discards candidates that can never answer a future query better than a newer candidate.

For next greater value, store indexes whose answer is unresolved. When `values[i]` is greater than the value at the top index, it resolves that index.

TEXT

```
values: 2  1  4
i=0: stack [0]
i=1: 1 is not greater than 2 -> [0,1]
i=2: 4 resolves index 1, then index 0 -> []
```

Every index is pushed once and popped at most once. That aggregate argument proves O(n) time even though a single iteration may pop many indexes.

Store indexes when the answer needs distance, boundaries, or access to both value and position. Store values only when position is irrelevant.

Monotonic deque for a window maximum

The deque stores indexes in decreasing value order:

1. remove front indexes outside the current window;
2. remove back indexes whose values are less than or equal to the new value;
3. append the new index;
4. the front is the maximum index.

Removing equal older values is safe when only the maximum value is required because the newer equal value expires later. If stable earliest-index behavior is required, adjust the comparison.

Boundary with PriorityQueue

A priority queue returns the globally best priority, not FIFO order and not arbitrary removal from both ends. Sliding-window maximum with lazy deletion can use heaps, but a monotonic deque is linear and keeps only undominated window candidates. Heaps are developed in the next dedicated volume.

Foundation failure clinic

- Popping before checking emptiness.
- Mixing front/back methods so the policy silently changes.
- Using `LinkedList` or `Stack` by habit without a needed contract.
- Marking BFS nodes visited too late.
- Storing values when indexes are required.
- Expiring window indexes after reporting the answer.
- Claiming a monotonic structure is sorted output; it stores only a useful candidate frontier.

TRY IT | Foundation checkpoint

1. Which ends implement a FIFO queue in `ArrayDeque`?
2. Why is `null` unavailable as an `ArrayDeque` element?
3. Why is next-greater processing O(n) rather than O(n squared)?
4. What exactly does a BFS queue contain?
5. When should equal values be removed from a monotonic deque?

SDE-2 CORE CHAPTER

Chapter 2 - Stacks, Queues, Deques, and Monotonic Patterns for SDE-2

These structures encode an ordering policy. A stack says the newest unfinished item is resolved first. A queue says work is resolved in discovery order. A deque permits controlled access at both boundaries. Monotonic structures add a dominance rule: discard entries that can never influence a future answer. SDE-2 candidates should recognize the policy from the problem, prove why discarded state is irrelevant, and connect the in-memory algorithm to bounded queues, load shedding, and backpressure in production.

Choose the policy before the class

Prompt signal	Policy	Java default
nested delimiters, undo, expression operators	LIFO	`ArrayDeque<E>` as stack
level order, shortest unweighted steps, arrival order	FIFO	`ArrayDeque<E>` as queue
both ends, window candidates, 0-1 transitions	deque	`ArrayDeque<E>`
nearest greater/smaller boundary	monotonic stack	deque of indexes
best element in each moving window	monotonic deque	deque of indexes
producer may outrun consumer	bounded blocking/nonblocking queue	explicit capacity and rejection policy

Prefer `ArrayDeque` over legacy `Stack`. It has no synchronization overhead, exposes both stack and queue operations, and rejects `null`, preventing ambiguity with methods that use `null` for "empty." It is not thread-safe.

The ArrayDeque API without end confusion

Intent	Mutating method	Empty/full alternative	End
stack push	`push(e)` / `addFirst(e)`	`offerFirst(e)`	front
stack pop	`pop()` / `removeFirst()`	`pollFirst()` returns `null`	front
stack peek	`getFirst()`	`peekFirst()` returns `null`	front
queue enqueue	`addLast(e)`	`offerLast(e)`	back
queue dequeue	`removeFirst()`	`pollFirst()` returns `null`	front
inspect queue head	`getFirst()`	`peekFirst()` returns `null`	front

`ArrayDeque` grows and therefore does not express operational capacity. Use `ArrayBlockingQueue`, another bounded queue, or a domain-specific ring buffer when overload must be visible. Method names do not provide thread safety or backpressure by themselves.

Family 1: delimiter validation

Recognition and invariant

Use a stack when every closing token must match the most recent unmatched opening token. Scan left to right. Before processing position `i`, the stack contains exactly the unmatched openings in the processed prefix, oldest at the bottom and newest at the top. Push openings. On a close, reject if the stack is empty or its top has the wrong type. At the end, accept only if the stack is empty.

Dry-run `{[()]}`: stack states are `{`, `{[`, `{[(`, `{[`, `{`, empty. For `([)]`, `)` sees `[` on top and fails even though some `(` exists deeper; nesting order, not just counts, is the requirement.

Time is `O(n)`, space is `O(n)` in the all-opening case. Define how non-delimiter characters are treated. The sample ignores them; a parser might reject them or tokenize strings/comments first. Never validate source code correctly by scanning raw characters without lexical rules.

Family 2: expression evaluation

Expression questions test precedence, associativity, tokenization, unary operators, and failure contracts. Two common designs are operator/value stacks (shunting-yard style) and recursive descent. The standalone implementation below uses recursive descent with this grammar:

TEXT

```
expression := term (("+" | "-") term)*
term       := factor (("*" | "/") factor)*
factor     := ("+" | "-") factor | number | "(" expression ")"
```

The sample grammar deliberately defines `number` as one or more ASCII digits `0` through `9`. Accepting broader Unicode digits requires digit-value conversion rather than subtracting the ASCII character `'0'`.

The invariant of `parseExpression` is that it enters at the beginning of an expression and returns its value with the cursor at the first token not belonging to that expression. `parseTerm` consumes multiplication/division before the expression layer sees addition/subtraction, encoding precedence structurally. The loops combine from left to right, giving left associativity for subtraction and division.

Dry-run `2 + 3 * (4 - 1)`: expression first obtains term `2`. It sees `+`; the next term obtains factor `3`, sees `*`, then evaluates parenthesized expression `4-1` as `3`. The term becomes `9`; the outer expression returns `11`.

Parsing is `O(n)` time and `O(d)` call depth for nesting/unary depth `d`. Arithmetic overflow, division rounding, numeric format, maximum nesting, and error location are API decisions. The sample uses `long`, Java's truncation toward zero, and explicit malformed-input errors. It detects overflow while accumulating a numeric literal, but the expression operators and unary negation intentionally retain ordinary Java `long` wraparound; a strict-arithmetic API would use the corresponding exact operations throughout.

Family 3: MinStack and synchronized representations

A MinStack supports `push`, `pop`, `top`, and `min` in constant time. One design stores pairs `(value, minimumSoFar)`. Another maintains a value stack plus a minimum stack. When pushing a value less than or equal to the current minimum, also push it on minima. On pop, remove from minima when the values equal.

The `<=` is important. For pushes `3,2,2`, minima must contain both copies of `2`; after one `2` is popped, the remaining minimum is still `2`. The invariant is that the minima stack holds exactly those values that established or tied a minimum among the corresponding value-stack prefixes. Operations are `O(1)` time and total space `O(n)`.

In concurrent code, two deques must be updated atomically under one lock or confined to one thread. Thread-safe components used separately do not create a thread-safe compound invariant.

Family 4: a circular queue

A fixed-size ring buffer makes capacity explicit. Track `head`, `size`, and an array. The tail insertion index is `(head + size) % capacity`. On removal, read `head`, advance it modulo capacity, and decrement size.

The invariant is that logical element `j`, for `0 <= j < size`, resides at `(head+j) % capacity`; all other slots are outside the logical queue. Keeping `size` distinguishes full from empty when head and tail indexes coincide. With capacity three: offer `10,20,30`; poll `10` moves head; offer `40` wraps into the freed slot, while logical order remains `20,30,40`.

Each operation is `O(1)`, space `O(capacity)`. Decide whether full means return `false`, throw, block, drop newest, drop oldest, or spill elsewhere. That policy is more important operationally than modulo arithmetic.

Family 5: monotonic stacks

Next greater and daily temperatures

For each element, a next-greater problem asks which later item first dominates it. Keep indexes whose answer is unresolved in decreasing value order. When current value exceeds the value at the top index, the current index is that older entry's first greater answer; pop and resolve repeatedly, then push current.

Why is it the first? Every intervening value failed to exceed the older value, or the older index would already have been popped. Why can it be removed? Its one requested answer is now final. Every index is pushed once and popped at most once, so the nested `while` is aggregate `O(n)`, not `O(n^2)`.

Dry-run temperatures `[73,74,75,71,69,72,76,73]`: index `0` resolves at `1`; index `1` resolves at `2`. Indexes `2,3,4` accumulate; value `72` resolves `4` and `3`, but not `2`. Value `76` resolves `5` and `2`. Waiting days become `[1,1,4,2,1,1,0,0]`.

Use `<` versus `<=` according to "greater" versus "greater or equal." Store indexes when distance, expiration, or duplicate identity matters.

Largest rectangle in a histogram

Maintain indexes with nondecreasing heights. When a lower height arrives, pop height `h`; after popping, the new stack top is the nearest strictly lower boundary on the left, and current index is the first lower boundary on the right. Width is `right - left - 1`. A virtual trailing zero flushes remaining bars.

For `[2,1,5,6,2,3]`, arrival of height `2` at index `4` pops `6` (area `6*1`) and `5` (area `5*2=10`). That is the optimum. Each index enters and leaves once: `O(n)` time and `O(n)` space. Equality policy changes which duplicate bar carries the width but should not change the maximum if implemented consistently.

Family 6: monotonic deque for sliding maximum

The deque stores candidate indexes in decreasing value order and increasing index order. Before producing window ending at `right`:

1. remove the front if it lies before `right-k+1`;
2. remove back indexes whose values are no greater than the new value;
3. append `right`;
4. the front is the maximum once a full window exists.

Removing dominated backs is sound: the new value is at least as large and expires later, so the old candidate can never again be a maximum. For `[1,3,-1,-3,5,3,6,7]`, `k=3`, maxima are `[3,3,5,5,6,7]`. Index storage is mandatory because values alone do not reveal expiration.

Time is `O(n)` aggregate, space `O(k)`. Validate `1 <= k <= n`. For immutable inputs, returning values is fine; a streaming system should define timestamp/window semantics and out-of-order handling.

Family 7: BFS frontier and operational backpressure

Breadth-first search uses a FIFO queue because all nodes at distance `d` must be expanded before any node at `d+1`. The invariant is that when a vertex is dequeued, its recorded distance is the shortest unweighted distance from the source. Mark visited when enqueuing, not when dequeuing; otherwise multiple parents can enqueue the same vertex and inflate memory.

On a grid, each edge changes one coordinate. Starting distance zero, enqueue legal unvisited neighbors with distance plus one. The first time the target is discovered or dequeued is shortest because FIFO order never skips a smaller layer. Complexity is `O(V+E)`; on an `r*c` four-neighbor grid, `O(r*c)` time and space.

An algorithmic frontier can grow to `O(V)`. A service queue can grow until the process fails. Production queues need capacity, admission control, timeouts, cancellation, prioritization, fairness, and metrics. "Use a queue" is not a complete system design.

Complete Java 21 reference implementation

Compile and run with `java -ea StackQueueDequeSde2`.

JAVA (continued 1/8)

```java
import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Deque;
import java.util.NoSuchElementException;
import java.util.OptionalInt;

public final class StackQueueDequeSde2 {
    private StackQueueDequeSde2() {}

    public static boolean validDelimiters(String text) {
        if (text == null) throw new IllegalArgumentException("text is null");
        Deque<Character> openings = new ArrayDeque<>();
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (ch == '(' || ch == '[' || ch == '{') {
                openings.push(ch);
            } else if (ch == ')' || ch == ']' || ch == '}') {
                if (openings.isEmpty() || !matches(openings.pop(), ch)) return false;
            }
        }
        return openings.isEmpty();
    }

    private static boolean matches(char open, char close) {
        return open == '(' && close == ')'
                || open == '[' && close == ']'
                || open == '{' && close == '}';
    }

    public static long evaluate(String expression) {
        if (expression == null) throw new IllegalArgumentException("expression is null");
        Parser parser = new Parser(expression);
        long value = parser.expression();
        parser.spaces();
        if (!parser.done()) throw parser.error("unexpected token");
```

JAVA (continued 2/8)

```
        return value;
    }

    private static final class Parser {
        private final String input;
        private int at;
        Parser(String input) { this.input = input; }
        boolean done() { return at == input.length(); }
        IllegalArgumentException error(String message) {
            return new IllegalArgumentException(message + " at offset " + at);
        }
        void spaces() {
            while (!done() && Character.isWhitespace(input.charAt(at))) at++;
        }
        boolean take(char expected) {
            spaces();
            if (!done() && input.charAt(at) == expected) { at++; return true; }
            return false;
        }
        long expression() {
            long value = term();
            while (true) {
                if (take('+')) value += term();
                else if (take('-')) value -= term();
                else return value;
            }
        }
        long term() {
            long value = factor();
            while (true) {
                if (take('*')) value *= factor();
                else if (take('/')) {
                    long divisor = factor();
                    if (divisor == 0) throw error("division by zero");
                    value /= divisor;
```

JAVA (continued 3/8)

```
                } else return value;
            }
        }
        long factor() {
            spaces();
            if (take('+')) return factor();
            if (take('-')) return -factor();
            if (take('(')) {
                long value = expression();
                if (!take(')')) throw error("missing ')'");
                return value;
            }
            spaces();
            if (done() || input.charAt(at) < '0' || input.charAt(at) > '9') {
                throw error("expected number");
            }
            long value = 0;
            while (!done() && input.charAt(at) >= '0' && input.charAt(at) <= '9') {
                value = Math.addExact(Math.multiplyExact(value, 10), input.charAt(at++) - '0');
            }
            return value;
        }
    }

    public static final class MinStack {
        private final Deque<Integer> values = new ArrayDeque<>();
        private final Deque<Integer> minima = new ArrayDeque<>();
        public void push(int value) {
            values.push(value);
            if (minima.isEmpty() || value <= minima.peek()) minima.push(value);
        }
        public int pop() {
            if (values.isEmpty()) throw new NoSuchElementException("empty stack");
            int value = values.pop();
            if (value == minima.peek()) minima.pop();
```

JAVA (continued 4/8)

```
            return value;
        }
        public int top() {
            if (values.isEmpty()) throw new NoSuchElementException("empty stack");
            return values.peek();
        }
        public int min() {
            if (minima.isEmpty()) throw new NoSuchElementException("empty stack");
            return minima.peek();
        }
        public boolean isEmpty() { return values.isEmpty(); }
    }

    public static final class CircularIntQueue {
        private final int[] elements;
        private int head;
        private int size;
        public CircularIntQueue(int capacity) {
            if (capacity <= 0) throw new IllegalArgumentException("capacity must be positive");
            elements = new int[capacity];
        }
        public boolean offer(int value) {
            if (size == elements.length) return false;
            elements[(head + size) % elements.length] = value;
            size++;
            return true;
        }
        public OptionalInt poll() {
            if (size == 0) return OptionalInt.empty();
            int value = elements[head];
            head = (head + 1) % elements.length;
            size--;
            return OptionalInt.of(value);
        }
```

JAVA (continued 5/8)

```
        public int size() { return size; }
    }

    public static int[] nextGreaterValues(int[] values) {
        if (values == null) throw new IllegalArgumentException("values is null");
        int[] answer = new int[values.length];
        Arrays.fill(answer, -1);
        Deque<Integer> unresolved = new ArrayDeque<>();
        for (int i = 0; i < values.length; i++) {
            while (!unresolved.isEmpty() && values[unresolved.peek()] < values[i]) {
                answer[unresolved.pop()] = values[i];
            }
            unresolved.push(i);
        }
        return answer;
    }

    public static int[] dailyTemperatures(int[] temperatures) {
        if (temperatures == null) throw new IllegalArgumentException("null input");
        int[] waits = new int[temperatures.length];
        Deque<Integer> unresolved = new ArrayDeque<>();
        for (int day = 0; day < temperatures.length; day++) {
            while (!unresolved.isEmpty()
                    && temperatures[unresolved.peek()] < temperatures[day]) {
                int older = unresolved.pop();
                waits[older] = day - older;
            }
            unresolved.push(day);
        }
        return waits;
    }

    public static long largestRectangle(int[] heights) {
        if (heights == null) throw new IllegalArgumentException("heights is null");
```

JAVA (continued 6/8)

```
        Deque<Integer> increasing = new ArrayDeque<>();
        long best = 0;
        for (int right = 0; right <= heights.length; right++) {
            int current = right == heights.length ? 0 : heights[right];
            if (current < 0) throw new IllegalArgumentException("negative height");
            while (!increasing.isEmpty() && heights[increasing.peek()] > current) {
                int height = heights[increasing.pop()];
                int left = increasing.isEmpty() ? -1 : increasing.peek();
                best = Math.max(best, (long) height * (right - left - 1));
            }
            if (right < heights.length) increasing.push(right);
        }
        return best;
    }

    public static int[] slidingMaximum(int[] values, int k) {
        if (values == null || k <= 0 || k > values.length) {
            throw new IllegalArgumentException("invalid window");
        }
        int[] answer = new int[values.length - k + 1];
        Deque<Integer> candidates = new ArrayDeque<>();
        for (int right = 0; right < values.length; right++) {
            int left = right - k + 1;
            while (!candidates.isEmpty() && candidates.peekFirst() < left) {
                candidates.removeFirst();
            }
            while (!candidates.isEmpty()
                    && values[candidates.peekLast()] <= values[right]) {
                candidates.removeLast();
            }
            candidates.addLast(right);
            if (left >= 0) answer[left] = values[candidates.peekFirst()];
        }
        return answer;
```

JAVA (continued 7/8)

```
    }

    public static int shortestGridPath(int[][] grid, int startRow, int startCol,
                                       int targetRow, int targetCol) {
        if (!inside(grid, startRow, startCol) || !inside(grid, targetRow, targetCol)
                || grid[startRow][startCol] != 0 || grid[targetRow][targetCol] != 0) {
            return -1;
        }
        boolean[][] seen = new boolean[grid.length][];
        for (int r = 0; r < grid.length; r++) seen[r] = new boolean[grid[r].length];
        ArrayDeque<int[]> queue = new ArrayDeque<>();
        queue.addLast(new int[] {startRow, startCol, 0});
        seen[startRow][startCol] = true;
        int[][] directions = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
        while (!queue.isEmpty()) {
            int[] state = queue.removeFirst();
            if (state[0] == targetRow && state[1] == targetCol) return state[2];
            for (int[] direction : directions) {
                int nr = state[0] + direction[0], nc = state[1] + direction[1];
                if (inside(grid, nr, nc) && grid[nr][nc] == 0 && !seen[nr][nc]) {
                    seen[nr][nc] = true;
                    queue.addLast(new int[] {nr, nc, state[2] + 1});
                }
            }
        }
        return -1;
    }

    private static boolean inside(int[][] grid, int row, int col) {
        return grid != null && row >= 0 && row < grid.length
                && grid[row] != null && col >= 0 && col < grid[row].length;
    }

    public static void main(String[] args) {
```

JAVA (continued 8/8)

```
        assert validDelimiters("call({x[2]})");
        assert !validDelimiters("([)]");
        assert evaluate("2 + 3 * (4 - 1)") == 11;
        assert evaluate("-(8 - 3) * 2") == -10;
        boolean nonAsciiDigitRejected = false;
        try {
            evaluate("\u0661");
        } catch (IllegalArgumentException expected) {
            nonAsciiDigitRejected = true;
        }
        assert nonAsciiDigitRejected;

        MinStack minStack = new MinStack();
        minStack.push(3); minStack.push(2); minStack.push(2);
        assert minStack.min() == 2 && minStack.pop() == 2 && minStack.min() == 2;

        CircularIntQueue queue = new CircularIntQueue(3);
        assert queue.offer(10) && queue.offer(20) && queue.offer(30);
        assert !queue.offer(99) && queue.poll().orElseThrow() == 10;
        assert queue.offer(40) && queue.size() == 3;

        assert Arrays.equals(nextGreaterValues(new int[] {2, 1, 2, 4, 3}),
                             new int[] {4, 2, 4, -1, -1});
        assert Arrays.equals(dailyTemperatures(
                new int[] {73, 74, 75, 71, 69, 72, 76, 73}),
                new int[] {1, 1, 4, 2, 1, 1, 0, 0});
        assert largestRectangle(new int[] {2, 1, 5, 6, 2, 3}) == 10;
        assert Arrays.equals(slidingMaximum(new int[] {1, 3, -1, -3, 5, 3, 6, 7}, 3),
                             new int[] {3, 3, 5, 5, 6, 7});

        int[][] grid = {{0, 0, 1}, {1, 0, 0}, {0, 0, 0}};
        assert shortestGridPath(grid, 0, 0, 2, 2) == 4;
    }
}
```

RECAP | Complexity summary

Algorithm	Time	Auxiliary space	Important qualification
delimiter validation	`O(n)`	`O(n)`	tokenization contract matters
expression parsing	`O(n)`	`O(d)`	nesting depth `d`; overflow policy separate
MinStack operation	`O(1)`	`O(n)` total	two representations form one invariant
circular queue operation	`O(1)`	`O(capacity)`	full-policy is contractual
next greater / temperatures	`O(n)`	`O(n)`	each index pushed/popped once
largest rectangle	`O(n)`	`O(n)`	use `long` for area
window maximum	`O(n)`	`O(k)`	deque stores unexpired candidate indexes
BFS	`O(V+E)`	`O(V)`	mark visited on enqueue

WATCH OUT | Edge cases and common mistakes

- Calling `pop`/`removeFirst` on empty throws; `pollFirst` returns `null`. Choose intentionally.
- `ArrayDeque` rejects null values and is neither bounded nor thread-safe.
- Delimiter counts alone cannot prove correct nesting.
- Expression evaluators need a grammar. Ad hoc "previous operator" code often mishandles unary minus and parentheses.
- `Math.negateExact(Long.MIN_VALUE)` would be needed for fully checked negation; unary minus can overflow.
- MinStack must preserve duplicate minima.
- A ring buffer with only head and tail needs another bit/slot convention to distinguish full from empty; tracking size is simpler.
- Monotonic comparisons (`<` versus `<=`) encode whether equality dominates. State the requested relation.
- Store indexes, not only values, when distance or expiration matters.
- Histogram area may overflow `int` even when each height and width fits.
- In BFS, marking on dequeue creates duplicate frontier entries; forgetting a visited rule can make cyclic graphs nonterminating.

TRY IT | Exercises with model checkpoints

Exercise 1: decode a nested repetition string

Decode input such as `3[a2[c]]`.

Checkpoint: push the prior string builder and repeat count on `[`, start a new frame, and on `]` pop and append the completed fragment the requested number of times. Validate balanced tokens, digits followed by `[`, a maximum output size, and integer overflow in counts. Complexity must include output length.

Exercise 2: postfix evaluation

Evaluate whitespace-tokenized Reverse Polish Notation.

Checkpoint: push numbers; on an operator pop right operand first, then left. Reject insufficient and leftover operands. `8 3 -` is `5`, not `-5`. Define division and overflow behavior.

Exercise 3: queue from two stacks

Implement FIFO behavior using input and output stacks.

Checkpoint: enqueue pushes to input. Dequeue transfers all input items to output only when output is empty. Each item moves at most once between stacks, proving amortized `O(1)` operations even though one dequeue can be `O(n)`.

Exercise 4: next smaller boundaries

For every histogram bar, compute nearest smaller index left and right in two passes.

Checkpoint: use consistent strict/non-strict comparisons for duplicates, then compute width. Compare memory and clarity against the one-pass sentinel solution.

Exercise 5: 0-1 BFS

Find shortest paths when edge weights are only zero or one.

Checkpoint: relax a zero-weight neighbor at the deque front and a one-weight neighbor at the back. The deque maintains nondecreasing tentative distance. Discuss why ordinary BFS fails for weight one mixed with zero and why Dijkstra is more general.

Exercise 6: bounded work queue design

Design a request-processing queue for a service whose workers can process 500 tasks per second.

Checkpoint: specify capacity in time or task units, admission timeout, overload response, cancellation, retry ownership, fairness, and metrics for depth/age/rejection. A larger queue can increase latency rather than capacity. Little's Law and measured service time guide the budget.

SDE-2 production follow-ups

How does a BFS queue relate to system backpressure? Both hold discovered work awaiting service, but graph search owns a finite known state space while production arrivals may be unbounded. A service needs capacity and an explicit overload signal to upstream callers.

Can `ArrayDeque` be shared between threads? Not safely without external synchronization. Choose a concurrent queue whose semantics match the operation; even then, compound checks such as "if absent then enqueue" may need stronger coordination.

How do you make parsers robust? Cap input length and nesting, return structured error positions, avoid quadratic substring construction, define Unicode/token rules, and use exact arithmetic or a documented overflow policy. Fuzz malformed inputs and ensure failure consumes bounded time and memory.

What should be observable? Queue depth, oldest-item age, enqueue/dequeue rate, rejection count, service latency, timeout/cancellation rate, and saturation duration. For algorithmic monotonic structures, instrument input size and operation count when investigating performance; aggregate linearity should be visible.

Interview follow-up chain and model answers

Why does a nested `while` in a monotonic stack remain linear? Charge work to entries, not loop nesting. Every index is pushed exactly once. Once popped, it never re-enters. Across the entire scan there are at most `n` pushes and `n` pops, so total deque operations are `O(n)` even if one input element triggers many pops.

When should equal values be removed from a monotonic deque? For window maximum, removing an older equal value is safe because the newer equal value produces the same maximum and expires later. This keeps the deque smaller. If the contract asks for the earliest index of a maximum, retain equals instead. The comparison embodies a tie-breaking contract, not only a micro-optimization.

Why mark BFS vertices when enqueued? Enqueueing is the moment a shortest layer first discovers the vertex. Marking then prevents every other same/next-layer parent from adding duplicates. Marking only on removal may preserve distance correctness but can inflate the queue dramatically, and without careful handling can repeat work.

What makes queue backpressure different from a lock? A lock coordinates exclusive access to state. Backpressure limits outstanding work and signals that downstream capacity is exhausted. A thread-safe unbounded queue solves race conditions but can still consume memory until failure. A bounded queue needs a full policy that propagates overload rather than merely hiding it.

How would you make MinStack persistent? Store immutable nodes containing value, minimum-so-far, and previous. Each push returns a new head, and pop returns the previous head, so versions share tails safely. Operations remain `O(1)` and snapshots are cheap, at the cost of one allocation per push and eventual garbage collection.

Can sliding-window maximum handle an online stream? Yes, if each item has a monotonically increasing sequence or timestamp. Evict front candidates outside the current window and remove dominated backs. Time windows also need a policy for late/out-of-order events and watermark progress; count windows do not. A production operator should expose state size, lag, and late-event counts.

A complete pattern-selection checkpoint

When a prompt says "nearest," do not immediately choose a heap. Ask whether the answer must respect sequence position. A heap retrieves a global extreme but does not efficiently discard an arbitrary expired element. A monotonic deque retains only nondominated candidates in positional order. When a prompt says "minimum number of transitions," inspect edge weights: FIFO BFS proves shortest paths only when every edge costs the same; 0-1 BFS uses a deque, and general nonnegative weights require a priority queue. When a prompt says "process tasks," separate algorithmic order from operational policy: FIFO may be fair enough for a finite graph, but a service may need priorities, tenant fairness, deadlines, idempotency, and poison-task handling. This selection conversation demonstrates more senior judgment than reciting container APIs.

Final readiness checklist

- I choose LIFO, FIFO, or two-ended access from the ordering requirement.
- I can use `ArrayDeque` without mixing front and back conventions.
- My stack/queue invariant explains every stored entry.
- I prove aggregate linear monotonic work by push/pop counts.
- I store indexes when age, distance, or expiration matters.
- BFS marks on enqueue and states the unweighted-edge assumption.
- Capacity, overload, cancellation, and concurrency are explicit production policies.

The transferable skill is not knowing that a deque exists. It is seeing which unresolved states deserve to remain and proving when an old state can never matter again.

SDE-2 CORE CHAPTER

Chapter 3 - Essential Monotonic-Structure Pattern Clinics

Next-greater and sliding-window maximum introduce monotonic structures. Histogram area and trapped rain water test whether the reader can define what a popped index means and compute its boundaries without guessing.

Clinic 1: largest rectangle in a histogram

Maintain indexes of bars whose heights are nondecreasing. A shorter current bar proves that every taller bar on top can no longer extend right. When height index `middle` is popped:

- the current index `right` is the first strictly shorter bar to its right;
- the new stack top is the nearest shorter bar to its left;
- width is `right` when the stack is empty, otherwise `right - stack.peek() - 1`.

For `[2, 1, 5, 6, 2, 3]`, index 4 with height 2 closes heights 6 and 5. Height 5 spans indexes 2 and 3, producing area 10.

A virtual zero-height bar at the end flushes every unresolved bar. Area uses `long` because `height * width` can overflow `int` under a large-input contract.

Duplicate policy

Keeping equal heights or collapsing them can both be correct, but the width proof must match the chosen comparison. This implementation pops only strictly taller bars, allowing the earlier equal bar to inherit the wider span later.

Clinic 2: trapped rain water with a monotonic stack

Maintain indexes in nonincreasing height order. When the current bar is taller than the top, the popped bar becomes a basin bottom. If the stack is then empty, no left boundary exists. Otherwise:

TEXT

```
distance = currentIndex - leftBoundaryIndex - 1
boundedHeight = min(leftHeight, currentHeight) - bottomHeight
water += distance * boundedHeight
```

Each index is pushed once and popped at most once, so the total is O(n), not O(n squared), despite the nested loop.

A two-pointer solution uses O(1) auxiliary space and is often simpler once the left-max/right-max invariant is understood. The stack version is valuable because it generalizes the boundary-closing idea used by histogram and next-greater problems.

Runnable Java 21 clinic

JAVA (continued 1/2)

```java
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.Objects;

public final class MonotonicCoverageClinic {
    private MonotonicCoverageClinic() {
    }

    public static long largestRectangle(int[] heights) {
        Objects.requireNonNull(heights, "heights");
        Deque<Integer> increasing = new ArrayDeque<>();
        long best = 0;

        for (int right = 0; right <= heights.length; right++) {
            int currentHeight = right == heights.length ? 0 : heights[right];
            if (currentHeight < 0) {
                throw new IllegalArgumentException("heights must be nonnegative");
            }
            while (!increasing.isEmpty()
                    && heights[increasing.peek()] > currentHeight) {
                int height = heights[increasing.pop()];
                int width = increasing.isEmpty()
                        ? right
                        : right - increasing.peek() - 1;
                best = Math.max(best, (long) height * width);
            }
            if (right < heights.length) {
                increasing.push(right);
            }
        }
        return best;
    }
```

JAVA (continued 2/2)

```
    public static long trappedWater(int[] heights) {
        Objects.requireNonNull(heights, "heights");
        Deque<Integer> decreasing = new ArrayDeque<>();
        long water = 0;

        for (int right = 0; right < heights.length; right++) {
            if (heights[right] < 0) {
                throw new IllegalArgumentException("heights must be nonnegative");
            }
            while (!decreasing.isEmpty()
                    && heights[right] > heights[decreasing.peek()]) {
                int bottom = decreasing.pop();
                if (decreasing.isEmpty()) {
                    break;
                }
                int left = decreasing.peek();
                int distance = right - left - 1;
                int boundedHeight = Math.min(heights[left], heights[right])
                        - heights[bottom];
                water += (long) distance * boundedHeight;
            }
            decreasing.push(right);
        }
        return water;
    }

    public static void main(String[] args) {
        assert largestRectangle(new int[] {2, 1, 5, 6, 2, 3}) == 10;
        assert largestRectangle(new int[0]) == 0;
        assert trappedWater(new int[] {0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1}) == 6;
        System.out.println("PASS essential monotonic clinics");
    }
}
```

Expected output with assertions enabled:

TEXT

```
PASS essential monotonic clinics
```

Interviewer follow-up chain with model answers

Interviewer: Why store indexes rather than heights?

Candidate: Width depends on positions, and duplicates need separate boundary histories. An index also gives the height, so it preserves both pieces of information.

Interviewer: Why is the histogram algorithm linear?

Candidate: Every index enters once and leaves at most once. The while-loop work aggregates to at most n pops across the complete scan.

Interviewer: Which rain-water solution would you code in an interview?

Candidate: If I can explain the left-max/right-max invariant cleanly, I would use two pointers for $O(1)$ space. I would choose the stack when the interviewer wants a boundary-closing monotonic formulation or when extending to related basin events.

SDE-2 CORE CHAPTER

Chapter 4 - Realistic Stack, Queue, and Deque Interview Rounds

Round 1: daily temperatures

Prompt

For each day's temperature, return how many days must pass until a strictly warmer temperature. Return zero when none exists.

Candidate derivation

Brute force scans right from every day: O(n squared). Instead, keep indexes of unresolved days in decreasing temperature order. A warmer current day resolves every colder index at the top.

JAVA

```java
static int[] daysUntilWarmer(int[] temperatures) {
    int[] answer = new int[temperatures.length];
    Deque<Integer> unresolved = new ArrayDeque<>();
    for (int day = 0; day < temperatures.length; day++) {
        while (!unresolved.isEmpty()
                && temperatures[day] > temperatures[unresolved.peek()]) {
            int earlier = unresolved.pop();
            answer[earlier] = day - earlier;
        }
        unresolved.push(day);
    }
    return answer;
}
```

Why indexes? The output is a distance. The temperature can be recovered from the array.

Why strictly greater? Equal temperature does not satisfy "warmer," so equality stays unresolved.

Complexity? O(n) time by push/pop aggregate analysis and O(n) auxiliary space.

Round 2: sliding-window maximum

Prompt and clarification

Return the maximum for every contiguous window of size `k`. Require `1 <= k <= n`.

Model answer

JAVA

```java
static int[] windowMaximum(int[] values, int k) {
    if (k < 1 || k > values.length) {
        throw new IllegalArgumentException("invalid window");
    }
    int[] answer = new int[values.length - k + 1];
    Deque<Integer> candidates = new ArrayDeque<>();
    for (int right = 0; right < values.length; right++) {
        int left = right - k + 1;
        while (!candidates.isEmpty() && candidates.peekFirst() < left) {
            candidates.removeFirst();
        }
        while (!candidates.isEmpty()
                && values[candidates.peekLast()] <= values[right]) {
            candidates.removeLast();
        }
        candidates.addLast(right);
        if (left >= 0) {
            answer[left] = values[candidates.peekFirst()];
        }
    }
    return answer;
}
```

Invariant: candidate indexes are inside the active window, increase from front to back, and their values strictly decrease. Any removed-back index is dominated by the newer index for all future windows containing both.

Follow-up answers

Could a heap work? Yes, O(n log n) with lazy expiry or indexed deletion. The deque exploits ordered window expiry and dominance for O(n).

Streaming? Yes. Emit a maximum after receiving the first k values. Retain indexes or sequence numbers for expiry.

Round 3: queue using two stacks

Prompt

Implement FIFO queue operations with two stacks and explain amortized cost.

Candidate answer

Push new elements onto `incoming`. Pop or peek from `outgoing`. When `outgoing` is empty, move every incoming element to it, reversing the order once.

JAVA

```java
static final class TwoStackQueue<T> {
    private final Deque<T> incoming = new ArrayDeque<>();
    private final Deque<T> outgoing = new ArrayDeque<>();

    void add(T value) {
        incoming.push(Objects.requireNonNull(value));
    }

    T remove() {
        transferIfNeeded();
        return outgoing.pop();
    }

    T peek() {
        transferIfNeeded();
        return outgoing.peek();
    }

    boolean isEmpty() {
        return incoming.isEmpty() && outgoing.isEmpty();
    }

    private void transferIfNeeded() {
        if (outgoing.isEmpty()) {
            while (!incoming.isEmpty()) {
                outgoing.push(incoming.pop());
            }
        }
    }
}
```

Each element is pushed to incoming, moved at most once, and popped from outgoing. A single remove can cost O(n), but a sequence of operations costs O(1) amortized per operation.

SDE-2 follow-ups

Can it be bounded? Track total size and reject or block adds at capacity. Blocking adds introduce concurrency, interruption, and condition signaling beyond the DSA contract.

Is it thread-safe? No. Thread safety requires a synchronization design; choosing concurrent collections is a separate production decision.

Closing answer pattern

Name the access policy, the exact deque ends, the stored state, the invariant, empty behavior, amortized reasoning, and one API or concurrency boundary.

SDE-2 CORE CHAPTER

Chapter 5 - Circular Deque and Monotonic Invariants

Stack, queue, and deque are access contracts over ordered state. A stack uses one end, a queue uses opposite ends, and a deque exposes both. Java's `ArrayDeque` is the right default for interview solutions, but implementing a circular deque once makes wraparound, resizing, and monotonic candidate structures much easier to reason about.

The complete primitive deque, monotonic algorithms, and executable checks are in `OrderingStructuresInterviewChecks.java`.

One buffer, logical order, physical wraparound

The companion stores:

- `head`: physical index of the first logical value;
- `size`: number of logical values; and
- `elements.length`: capacity.

Logical offset `k` lives at:

```text
(head + k) % capacity
```

Example:

```text
physical indexes:  0   1   2   3   4   5   6   7
buffer:            C   D   .   .   .   A   B   .
head = 5, size = 4
logical order:     A, B, C, D
```

The array contents outside logical size are irrelevant. Empty and full cannot both be inferred from `head == tail`; storing `size` removes that ambiguity.

Operations at both ends

- `addFirst`: decrement head with wraparound, write, increment size.
- `addLast`: write at logical offset `size`, increment size.
- `removeFirst`: read head, increment head, decrement size.
- `removeLast`: read logical offset `size - 1`, decrement size.

All are `O(1)` except a resize. On resize, copy logical offsets `0..size-1` into a larger array beginning at physical zero and reset `head = 0`. Copying physical indexes directly would scramble wrapped order.

Geometric doubling makes repeated end insertion amortized `O(1)`. The companion starts with nonzero capacity so doubling cannot remain zero.

Stack and queue views

With `ArrayDeque<E>`:

```text
stack: push / pop / peek       operate at first end
queue: addLast / removeFirst   opposite ends
deque: add/remove at either end
```

Avoid legacy `Stack` for new interview code. Also remember `ArrayDeque` rejects null, allowing null-returning methods to represent emptiness unambiguously. Choose throwing versus sentinel APIs deliberately; the custom primitive deque throws `NoSuchElementException` on empty removal.

Monotonic stack: unresolved candidates

For "days until a warmer temperature," the stack stores indexes whose answer is unresolved. Temperatures at those indexes are nonincreasing from bottom to top. A warmer current day resolves every smaller top:

```text
temperatures: 73, 74, 75, 71, 69, 72
day 4 stack:  [75@2, 71@3, 69@4]
day 5 = 72:   pop 69 -> wait 1
              pop 71 -> wait 2
              stop below 75
```

Each index is pushed once and popped at most once, so the nested-looking while loop is `O(n)` total.

Store indexes when distance, boundaries, or original position matters. Store values only when identity and position truly do not matter.

Monotonic deque: best candidate in a sliding window

For window maximum, candidate indexes are:

1. inside the current window; and
2. strictly decreasing by value from front to back.

Before adding `right`, remove expired indexes from the front and remove values no better than the new value from the back. The front is then the maximum.

Removing equal older values with `<=` is safe for maximum values: the newer equal value lasts longer. If the output needs the earliest maximum index, equality policy changes.

Histogram area: boundaries arrive when height drops

An increasing stack delays a bar until a shorter bar reveals its right boundary. When height at index `i` is lower than stack top:

```text
height = popped bar height
right boundary = i (exclusive)
left smaller index = new stack top, or -1
width = i - leftSmaller - 1
area = (long) height * width
```

A virtual trailing height zero flushes remaining bars. Area uses `long`; two `Integer.MAX_VALUE` bars already exceed `int`.

Postfix evaluation: operand order matters

For a binary operator, pop right operand first and left operand second:

```text
tokens: 7 3 -
right = 3, left = 7, result = 4
```

Reversing pop order is invisible for addition but wrong for subtraction and division. The companion validates stack arity, final stack size, numeric tokens, division by zero, and arithmetic overflow.

Edge-case matrix

Case	Correct handling	Common failure
empty deque removal	explicit exception/sentinel contract	reading stale buffer slot
wrapped resize	copy logical order, reset head	copy physical slices incorrectly
capacity one	grow before second insert	overwrite live value
size returns to zero	normalize or maintain valid head	confusing old indexes with live state
duplicate window maxima	choose equality/tie contract	wrong retained index
`k=1`	each value is its own maximum	off-by-one output size
`k>n` or zero	reject	negative allocation
equal histogram heights	consistent pop policy	double-count/wrong boundary
area overflow	multiply in `long`	overflow before assignment
postfix subtraction/division	pop right then left	reversed result
malformed expression	require two operands and one final result	accepting unused operands
mutable queued priority/state	use immutable snapshots or controlled updates	order silently stale

Six live interview Q&A chains

1. Full versus empty ring

Interviewer: Why keep `size`?

Candidate: With only equal head/tail indexes, full and empty can look identical unless one slot is sacrificed. `size` gives an explicit invariant and lets every capacity slot hold data.

2. Resize correctness

Interviewer: Can you call `Arrays.copyOf` on the wrapped buffer?

Candidate: It preserves physical order, not logical order. I copy `elements[(head+k)%oldCapacity]` to new index `k`, then set head to zero.

3. Nested monotonic loop

Interviewer: Why is the temperature solution not quadratic?

Candidate: Every index enters once and leaves at most once. Aggregate pops over the whole scan are at most `n`, so total work is linear.

4. Window equality

Interviewer: Why discard an older equal maximum?

Candidate: For a value-only answer, the newer equal index dominates: same value, later expiration. If the API requires earliest-max identity, I retain the older equal index by changing `<=` to `<`.

5. Histogram sentinel

Interviewer: Why append a zero-height bar?

Candidate: It supplies a right boundary for every still-open positive bar, letting one loop handle cleanup. I model it virtually so input is not modified.

6. Standard versus custom deque

Interviewer: Would you use your custom deque in the interview solution?

Candidate: Only if implementation is the question. For algorithm problems I use `ArrayDeque`, state the end operations clearly, and focus on the monotonic invariant. The custom version demonstrates that I understand the mechanics beneath the API.

Run the companion

BASH

```
javac --release 21 -Xlint:all -Werror OrderingStructuresInterviewChecks.java
java OrderingStructuresInterviewChecks
```

Expected final line: `PASS 16 ordering-structure checks`.

SDE-2 CORE CHAPTER

Chapter 6 - Amortized Analysis

Why this chapter exists

Several structures in this volume have an operation whose *worst case* is O(n) while the structure as a whole is still considered O(1) per operation. The monotonic stack pops an unbounded number of elements in one step. A queue built from two stacks moves the entire contents on one dequeue. `ArrayDeque` and `ArrayList` copy everything when they grow.

Candidates who say "the inner while loop makes this O(n^2)" about a monotonic stack are reading the code correctly and analysing it wrongly. **Amortized analysis is what makes the correct answer sayable**, and interviewers ask for it directly - "you have a nested loop there, what is the complexity?" is a test, not a trap.

Three techniques answer it, and the middle one is worth genuinely understanding rather than reciting.

What amortized does and does not mean

Amortized O(1) means: any sequence of `n` operations costs O(n) total, so the average per operation is O(1). It is a guarantee about the *sequence*, and it holds for every sequence, including adversarial ones.

It is not average-case analysis. Average case assumes a distribution over inputs and can be defeated by a hostile one. Amortized makes no probabilistic assumption at all - it is a worst-case bound on the total.

That distinction is the one interviewers probe, and stating it correctly separates understanding from vocabulary:

	Guarantee about	Defeated by an adversary?
Worst case per operation	one operation	no
Amortized	a sequence of operations	**no**
Average case	a distribution of inputs	yes

The practical caveat: amortized bounds say nothing about **latency**. A single operation can still take O(n), and for a real-time or low-tail-latency system that spike matters. `ArrayList.add` is amortized O(1) and can still stall on a resize of a large list. Naming that caveat unprompted is a strong senior signal.

Technique 1: aggregate analysis

Bound the total work over `n` operations directly, then divide.

Monotonic stack. Computing the next greater element pushes each index once and pops it at most once. The inner `while` may pop many elements in one iteration, but across the whole run **the total number of pops cannot exceed the total number of pushes, which is** `n`. Total work O(n), amortized O(1) per element.

JAVA

```java
static int[] nextGreater(int[] values) {
    int[] answer = new int[values.length];
    Arrays.fill(answer, -1);
    Deque<Integer> stack = new ArrayDeque<>();          // holds indices
    for (int i = 0; i < values.length; i++) {
        while (!stack.isEmpty() && values[stack.peek()] < values[i]) {
            answer[stack.pop()] = values[i];            // each index pops once, ever
        }
        stack.push(i);
    }
    return answer;
}
```

The sentence that answers the interviewer is: *"the inner loop is bounded not by n per iteration but by the number of pushes remaining, and each index is pushed exactly once and popped at most once, so the total is O(n)."*

Dynamic array growth. Doubling on resize, `n` appends cost `1 + 2 + 4 + ... + n < 2n` copies. Total O(n), amortized O(1).

The doubling factor is what makes this work. **Growing by a constant amount instead is O(n^2)**: growing by 10 requires `n/10` resizes copying an average of `n/2` each. Any *multiplicative* factor gives amortized O(1) - the factor changes the constant and the memory overhead, not the class.

Technique 2: the accounting method

Aggregate analysis needs you to see the total. The accounting method is more mechanical: **overcharge cheap operations and store the surplus as credit, then spend it on expensive ones.** If credit never goes negative, the charged rate is a valid amortized bound.

Queue from two stacks. `inbox` receives pushes; `outbox` serves pops. When `outbox` empties, everything transfers.

JAVA

```java
public final class QueueFromStacks<T> {
    private final Deque<T> inbox = new ArrayDeque<>();
    private final Deque<T> outbox = new ArrayDeque<>();

    public void enqueue(T item) {
        inbox.push(item);
    }

    public T dequeue() {
        if (outbox.isEmpty()) {
            while (!inbox.isEmpty()) {
                outbox.push(inbox.pop());     // O(n) - but only once per element
            }
        }
        if (outbox.isEmpty()) {
            throw new NoSuchElementException("empty queue");
        }
        return outbox.pop();
    }
}
```

A single `dequeue` can be O(n). The accounting argument:

TEXT

```
charge enqueue 3 credits:  1 pays the push onto inbox
                           1 saved for the future pop from inbox
                           1 saved for the future push onto outbox
charge dequeue 1 credit:   pays the pop from outbox
```

Every element is moved at most once from `inbox` to `outbox`, and the two credits banked at enqueue pay for exactly that move. Credit never goes negative, so **enqueue is amortized O(1) and dequeue is amortized O(1)**, even though one dequeue can be O(n).

The transfer condition matters more than it first appears, and it is worth being exact about why.

Refilling only when `outbox` is empty is what guarantees each element moves once, which is what the accounting argument depends on. Transferring on *every* dequeue is the standard bug - and its first symptom is not slowness but **wrong output**. If `outbox` still holds older elements, newly transferred ones land on top of them and are dequeued first, so the queue stops being FIFO. Running both versions over twenty thousand mixed operations, the always-transfer variant produced incorrect ordering while moving barely more elements than the correct one.

So the guard is a correctness control that also happens to preserve the amortized bound. That ordering matters: a candidate who defends it purely on performance grounds has not noticed the more serious failure.

Technique 3: the potential method

A more formal version: define a potential function over the structure's state, and let amortized cost be actual cost plus the change in potential.

```text
amortized(i) = actual(i) + potential(after) - potential(before)
```

For the two-stack queue, `potential = 2 * inbox.size()`. An enqueue adds one actual push and raises potential by 2, giving amortized 3. A dequeue that transfers `k` elements does `2k` actual work but drops potential by `2k`, giving amortized 1 plus the outbox pop.

This is the same argument as accounting, expressed as a function of state rather than as saved coins. It is worth naming - "the potential is the pending work stored in the inbox" - and rarely worth deriving formally in an interview. Accounting is easier to say out loud.

The one to watch: amortization does not survive everything

Two situations break these bounds, and knowing them is the depth question.

Repeated undo destroys amortization. Amortized bounds assume operations accumulate credit. Consider an `ArrayList` at exactly its capacity boundary: adding triggers a resize, removing shrinks back, and alternating add/remove at that boundary pays O(n) *every time* if the implementation shrinks eagerly. Real implementations avoid this with hysteresis - shrinking only at, say, one quarter full rather than one half - so a single operation cannot flip the state back. That gap between the shrink and grow thresholds is exactly what preserves the amortized bound.

Persistent or copied structures break it. If a caller can save a snapshot and repeatedly invoke the expensive operation from the same saved state, the banked credit is spent many times. Amortization assumes a single linear sequence of operations, and immutable or versioned structures violate that assumption.

WATCH OUT | Edge cases and common mistakes

- Calling a monotonic stack O(n^2) because of the inner `while`.
- Confusing amortized with average case; amortized holds against adversarial inputs.
- Assuming amortized O(1) bounds latency. A single operation can still take O(n).
- Growing a dynamic array by a constant rather than a factor, making appends O(n^2) overall.
- Transferring between the two stacks on every dequeue, which breaks FIFO order - newly moved elements sit on top of older ones.
- Forgetting the empty-queue check after the transfer, so an empty queue pops from an empty outbox.
- Shrinking a dynamic array at the same threshold it grows at, letting alternating add/remove cost O(n) each.
- Claiming amortized bounds hold for persistent structures where an expensive state can be replayed.
- Charging credit in the accounting method without checking it never goes negative.

TRY IT | Interview questions and model answers

Your monotonic stack has a nested while loop. Is it O(n^2)?

No, O(n) total. The inner loop is bounded by the number of elements currently on the stack, and each index is pushed exactly once and popped at most once across the whole run. So total pops are at most total pushes, which is n. Amortized O(1) per element.

What does amortized O(1) actually guarantee?

That any sequence of n operations costs O(n) in total, so the average is constant. It is a worst-case bound on the sequence and holds against adversarial inputs, which is what distinguishes it from average-case analysis. It does not bound any individual operation, so latency spikes remain possible.

Prove the two-stack queue is amortized O(1).

Accounting method. Charge each enqueue three credits: one for the push onto the inbox, and two banked for the eventual pop from the inbox and push onto the outbox. Each element moves between the stacks at most once, and the banked credits pay for exactly that. Credit never goes negative, so both operations are amortized O(1) despite one dequeue costing O(n).

Why must the transfer happen only when the outbox is empty?

Primarily for correctness. If the outbox still holds older elements, transferring puts newer ones on top of them, so they dequeue first and the queue is no longer FIFO. The guard also preserves the amortized bound, since it is what makes each element move exactly once - but the ordering failure is the more serious one and the one to lead with.

Why does doubling matter for dynamic arrays?

The total copy cost across n appends is a geometric series bounded by 2n, giving amortized O(1). Growing by a constant amount instead needs n/c resizes copying an average of n/2 each, which is O(n^2) overall. Any multiplicative factor works; the specific factor trades memory overhead against resize frequency.

When does an amortized bound stop holding?

When the sequence assumption breaks. Alternating operations at a resize boundary can cost O(n) each if the shrink and grow thresholds coincide, which is why implementations leave hysteresis between them. And persistent or snapshot-able structures break it entirely, because a caller can replay the expensive operation from a saved state and spend the same credit repeatedly.

TRY IT | Exercises

1. **Foundation:** Count total pushes and pops for `nextGreater` on a strictly decreasing array and on a strictly increasing one. Confirm both are O(n).
2. **Foundation:** Compute total copies for a thousand appends with doubling and with growth by ten. State both complexity classes.
3. **Interview Core:** Implement the two-stack queue and instrument it to report total element moves over ten thousand mixed operations. Compare against the operation count.
4. **Interview Core:** Change the transfer to happen on every dequeue, then check the output order against a reference queue before looking at the move count. Say which failure you would have caught first by reading the code.
5. **Interview Core:** Write the accounting argument for a dynamic array with doubling, stating the credit charged per append.
6. **Interview Core:** Build a stack with eager shrinking at half capacity and construct the alternating sequence that costs O(n) per operation.
7. **SDE-2 Follow-up:** Define the potential function for the two-stack queue and derive the amortized cost of both operations.
8. **SDE-2 Follow-up:** Measure worst single-operation latency for `ArrayList.add` over a million appends and reconcile it with the amortized bound.
9. **SDE-2 Follow-up:** Explain why a persistent version of the two-stack queue loses the amortized guarantee, with a concrete calling pattern.
10. **Challenge:** Design a stack supporting `push`, `pop`, and `getMin` in worst-case O(1) - not amortized - and state the extra memory it costs.

RECAP | Chapter summary

Several structures in this volume have an operation whose worst case is linear while the structure is still O(1) per operation overall, and amortized analysis is what makes that sayable. It guarantees that any sequence of n operations costs O(n) total, which is a worst-case bound on the sequence rather than an average over inputs - so it holds against adversarial data, and it says nothing about the latency of any single operation. Three techniques deliver it: aggregate analysis bounds the total directly, which is why a monotonic stack's inner loop is O(n) overall since each index is pushed once and popped at most once; the accounting method banks credit on cheap operations to pay for expensive ones, which proves the two-stack queue amortized O(1) provided the transfer happens only when the outbox empties - a guard that is first a correctness control, since transferring onto a non-empty outbox breaks FIFO order outright; and the potential method expresses the same argument as a function of state. The bounds break in two places worth knowing - resize thresholds without hysteresis, where alternating operations pay the full cost every time, and persistent structures, where a saved state lets the same banked credit be spent repeatedly.

TRY IT | Revision checklist

- [ ] I can explain why a monotonic stack's nested loop is O(n) total.
- [ ] I can distinguish amortized from average case and say which an adversary defeats.
- [ ] I know amortized bounds do not bound latency.
- [ ] I can give the accounting proof for the two-stack queue.
- [ ] I know the transfer guard is a correctness control first and an amortization control second.
- [ ] I can explain why doubling gives O(1) and constant growth gives O(n^2).
- [ ] I can state the potential function for the two-stack queue.
- [ ] I know why shrink and grow thresholds must differ.
- [ ] I know persistent structures break amortized guarantees.

SDE-2 CORE CHAPTER

Chapter 7 - Stack, Queue, and Deque Practice Lab

1 **Foundation:** Map `push`, `pop`, and `peek` to explicit deque ends.

2 **Foundation:** Explain the difference between `removeFirst` and `pollFirst` on empty input.

3 **Interview Core:** State the invariant for a delimiter-validation stack.

4 **Interview Core:** Why should BFS usually mark visited at enqueue time?

5 Predict the output after `addLast(1)`, `addLast(2)`, `removeLast()`.

6 Debug a queue that uses `addLast` and `removeLast`.

7 Debug a next-greater loop that stores values but later computes index distance.

8 Implement postfix-expression evaluation with malformed-input checks.

9 Implement a fixed-capacity circular queue.

10 Implement next smaller value to the right.

11 Solve largest rectangle in a histogram using sentinels or a final flush.

12 Implement a min stack with duplicate minima.

13 Explain why `PriorityQueue` iteration is not sorted output.

14 Compare a monotonic deque with a balanced tree for sliding maximum.

15 Define backpressure behavior for a bounded work queue.

16 **Interview Core:** Compute trapped rain water with a monotonic stack. For every popped basin bottom, identify the left boundary, right boundary, width, and bounded height.

Internal mechanics lab

17 **Foundation:** Given capacity eight, head six, and size four, map logical offsets to physical indexes.

18 **Interview Core:** Implement a resizable primitive circular deque with both-end operations and a nonzero initial capacity.

19 **Interview Core:** Resize a wrapped deque by copying logical order; show the bug produced by copying physical order directly.

20 **Interview Core:** Implement largest histogram rectangle with a virtual trailing zero and `long` area.

21 **Interview Core:** Implement postfix evaluation with operand-order, arity, division, token, and overflow validation.

22 **SDE-2 Follow-up:** Differential-test the custom deque against `ArrayDeque` after every randomized operation, including repeated wraparound and resize.

SDE-2 CORE CHAPTER

Chapter 8 - Stack, Queue, and Deque Practice Lab Solutions

1 `push` inserts at the front, `pop` removes front, and `peek` inspects front. Equivalent explicit calls are `addFirst`, `removeFirst`, and `peekFirst`.

2 `removeFirst` throws `NoSuchElementException`; `pollFirst` returns null. Choose according to the method contract.

3 The stack contains exactly unmatched opening delimiters from the processed prefix, in nesting order.

4 Enqueue-time marking ensures each node enters the frontier once. Dequeue-time marking can admit duplicates from multiple parents.

5 It prints or returns 2; using the same end for insert and remove creates LIFO behavior.

6 Replace removal with `removeFirst` to obtain FIFO order.

7 Store indexes. Values alone cannot recover distance and duplicate values are ambiguous.

8 Push operands. On an operator, require two values, pop right then left, apply with overflow/division policy, and push the result. Require one final value.

9 Use an array, head, size, and capacity. Insert at `(head + size) % capacity`, remove at head, and update head modulo capacity.

10 Scan left to right with unresolved indexes, popping when the current value is smaller, or scan right to left while discarding values not smaller. Define strictness for equality.

11 Maintain increasing bar indexes. When a shorter bar arrives, pop height and compute width between current index and the new top. Flush with a zero-height sentinel.

12 Store `(value, currentMinimum)` per entry or use a synchronized second stack. Duplicate minima must each have matching lifetime.

13 A heap guarantees only that the head is minimal. Its backing-array iteration order is heap order, not global sorted order; repeatedly poll a copy for sorted output.

14 A deque is O(n) total and specialized for FIFO window expiry plus maximum. A tree supports broader ordered queries in O(log k) per update and handles arbitrary deletions if counts are managed.

15 State whether producers block, reject, drop newest, drop oldest, or time out; define capacity, fairness, shutdown, metrics, and interruption behavior.

16 Keep indexes in nonincreasing-height order. A taller current bar pops a basin bottom. If no left boundary remains, no water is enclosed; otherwise width is `right-left-1` and bounded height is `min(height[left],height[right])-height[bottom]`. Every index is pushed and popped at most once, giving O(n) time and O(n) stack space. A two-pointer formulation can reduce auxiliary space to O(1).

Internal mechanics solutions

17 Physical index is `(head+offset)%capacity`: offsets 0,1,2,3 map from head six to indexes 6,7,0,1. Logical order crosses the physical array boundary without moving existing values.

18 Store nonzero buffer, head, and size. `addFirst` wraps head backward; `addLast` writes logical offset size; removals read offset zero or size-1 and decrement size. Empty removal throws by contract. Geometric resize gives amortized constant-time insertion.

19 Allocate double capacity and for each logical offset `k`, copy old `[(head+k)%oldLength]` to new `[k]`; reset head zero. Direct `Arrays.copyOf` would retain physical sequence `C,D,...,A,B`, not logical `A,B,C,D`.

20 Keep increasing indexes. A shorter height pops a bar, with right boundary current index and left-smaller boundary new top or -1; width is `right-left-1`. Iterate one virtual final zero to flush. Cast width/height to `long` before multiplication.

21 For operators require two operands, pop right then left, and use exact arithmetic plus explicit division-zero and `MIN/-1` checks. Parse other tokens as `long`. Require exactly one final stack value; leftover operands and missing operands are malformed.

22 Apply identical seeded operations to custom deque and `ArrayDeque`: add/remove/peek both ends. After every operation compare size, endpoints, and full logical iteration. A small starting capacity plus thousands of mixed operations forces wrap and multiple resizes; empty-failure cases are targeted separately.

SDE-2 CORE CHAPTER

Chapter 9 - Executable Deque and Monotonic-Structure Checks

This dependency-free Java 21 companion validates a resizable circular deque, wrap and resize boundaries, monotonic invariants, histogram spans, postfix contracts, and differential cases. A successful run prints PASS 16 ordering-structure checks.

JAVA (continued 1/9)

```java
import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.Deque;
import java.util.NoSuchElementException;
import java.util.Random;

public final class OrderingStructuresInterviewChecks {
    private OrderingStructuresInterviewChecks() {}

    /** Primitive resizable circular deque used to expose head/size mechanics. */
    static final class IntArrayDeque {
        private int[] elements = new int[1];
        private int head;
        private int size;

        int size() {
            return size;
        }

        int capacity() {
            return elements.length;
        }

        boolean isEmpty() {
            return size == 0;
        }

        void addFirst(int value) {
            ensureCapacity();
            head = decrement(head, elements.length);
            elements[head] = value;
            size++;
        }
```

JAVA (continued 2/9)

```
        void addLast(int value) {
            ensureCapacity();
            elements[physicalIndex(size)] = value;
            size++;
        }

        int peekFirst() {
            requireNotEmpty();
            return elements[head];
        }

        int peekLast() {
            requireNotEmpty();
            return elements[physicalIndex(size - 1)];
        }

        int removeFirst() {
            int value = peekFirst();
            head = increment(head, elements.length);
            size--;
            if (size == 0) {
                head = 0;
            }
            return value;
        }

        int removeLast() {
            int value = peekLast();
            size--;
            if (size == 0) {
                head = 0;
            }
            return value;
        }
```

JAVA (continued 3/9)

```
    int[] logicalOrder() {
        int[] result = new int[size];
        for (int offset = 0; offset < size; offset++) {
            result[offset] = elements[physicalIndex(offset)];
        }
        return result;
    }

    private int physicalIndex(int logicalOffset) {
        return (head + logicalOffset) % elements.length;
    }

    private void ensureCapacity() {
        if (size < elements.length) {
            return;
        }
        int[] expanded = new int[Math.multiplyExact(elements.length, 2)];
        for (int offset = 0; offset < size; offset++) {
            expanded[offset] = elements[physicalIndex(offset)];
        }
        elements = expanded;
        head = 0;
    }

    private void requireNotEmpty() {
        if (size == 0) {
            throw new NoSuchElementException("empty deque");
        }
    }

    private static int increment(int index, int length) {
        return index + 1 == length ? 0 : index + 1;
    }
```

JAVA (continued 4/9)

```
        private static int decrement(int index, int length) {
            return index == 0 ? length - 1 : index - 1;
        }
    }

    static int[] daysUntilWarmer(int[] temperatures) {
        int[] answer = new int[temperatures.length];
        Deque<Integer> unresolved = new ArrayDeque<>();
        for (int day = 0; day < temperatures.length; day++) {
            while (!unresolved.isEmpty()
                    && temperatures[day] > temperatures[unresolved.peek()]) {
                int earlier = unresolved.pop();
                answer[earlier] = day - earlier;
            }
            unresolved.push(day);
        }
        return answer;
    }

    static int[] windowMaximum(int[] values, int k) {
        if (k < 1 || k > values.length) {
            throw new IllegalArgumentException("invalid window");
        }
        int[] answer = new int[values.length - k + 1];
        Deque<Integer> candidates = new ArrayDeque<>();
        for (int right = 0; right < values.length; right++) {
            int left = right - k + 1;
            while (!candidates.isEmpty() && candidates.peekFirst() < left) {
                candidates.removeFirst();
            }
            while (!candidates.isEmpty()
                    && values[candidates.peekLast()] <= values[right]) {
                candidates.removeLast();
```

JAVA (continued 5/9)

```
            }
            candidates.addLast(right);
            if (left >= 0) {
                answer[left] = values[candidates.peekFirst()];
            }
        }
        return answer;
    }

    static long largestRectangleArea(int[] heights) {
        Deque<Integer> increasing = new ArrayDeque<>();
        long best = 0;
        for (int index = 0; index <= heights.length; index++) {
            int currentHeight = index == heights.length ? 0 : heights[index];
            if (currentHeight < 0) {
                throw new IllegalArgumentException("heights cannot be negative");
            }
            while (!increasing.isEmpty()
                    && heights[increasing.peek()] > currentHeight) {
                int height = heights[increasing.pop()];
                int leftSmaller = increasing.isEmpty() ? -1 : increasing.peek();
                long width = index - (long) leftSmaller - 1L;
                best = Math.max(best, height * width);
            }
            if (index < heights.length) {
                increasing.push(index);
            }
        }
        return best;
    }

    static long evaluatePostfix(String[] tokens) {
        Deque<Long> operands = new ArrayDeque<>();
```

JAVA (continued 6/9)

```
        for (String token : tokens) {
            if (token.equals("+") || token.equals("-")
                    || token.equals("*") || token.equals("/")) {
                if (operands.size() < 2) {
                    throw new IllegalArgumentException("operator needs two operands");
                }
                long right = operands.pop();
                long left = operands.pop();
                long result = switch (token) {
                    case "+" -> Math.addExact(left, right);
                    case "-" -> Math.subtractExact(left, right);
                    case "*" -> Math.multiplyExact(left, right);
                    case "/" -> {
                        if (right == 0 || left == Long.MIN_VALUE && right == -1) {
                            throw new ArithmeticException("invalid division");
                        }
                        yield left / right;
                    }
                    default -> throw new AssertionError("unreachable operator");
                };
                operands.push(result);
            } else {
                try {
                    operands.push(Long.parseLong(token));
                } catch (NumberFormatException invalid) {
                    throw new IllegalArgumentException("invalid token: " + token, invalid);
                }
            }
        }
        if (operands.size() != 1) {
            throw new IllegalArgumentException("expression must produce one value");
        }
        return operands.pop();
```

JAVA (continued 7/9)

```
    }

    private static boolean circularDequeMatchesArrayDeque() {
        Random random = new Random(53L);
        IntArrayDeque custom = new IntArrayDeque();
        Deque<Integer> expected = new ArrayDeque<>();
        for (int operation = 0; operation < 10_000; operation++) {
            int choice = random.nextInt(6);
            if (expected.isEmpty() || choice < 2) {
                int value = random.nextInt();
                if ((choice & 1) == 0) {
                    custom.addFirst(value);
                    expected.addFirst(value);
                } else {
                    custom.addLast(value);
                    expected.addLast(value);
                }
            } else if (choice == 2) {
                if (custom.removeFirst() != expected.removeFirst()) {
                    return false;
                }
            } else if (choice == 3) {
                if (custom.removeLast() != expected.removeLast()) {
                    return false;
                }
            } else if (choice == 4 && custom.peekFirst() != expected.peekFirst()) {
                return false;
            } else if (choice == 5 && custom.peekLast() != expected.peekLast()) {
                return false;
            }
            if (custom.size() != expected.size()) {
                return false;
            }
```

JAVA (continued 8/9)

```
            int index = 0;
            int[] logical = custom.logicalOrder();
            for (int value : expected) {
                if (logical[index++] != value) {
                    return false;
                }
            }
        }
        return true;
    }

    private static void expectFailure(Runnable action, Class<? extends Throwable> type) {
        try {
            action.run();
        } catch (Throwable failure) {
            if (type.isInstance(failure)) {
                return;
            }
            throw new AssertionError("wrong failure type", failure);
        }
        throw new AssertionError("expected " + type.getSimpleName());
    }

    private static void check(boolean condition, String message) {
        if (!condition) {
            throw new AssertionError(message);
        }
    }

    public static void main(String[] args) {
        check(Arrays.equals(daysUntilWarmer(new int[] {73, 74, 75, 71, 69, 72, 76, 73}),
                new int[] {1, 1, 4, 2, 1, 1, 0, 0}), "temperatures");
        check(Arrays.equals(windowMaximum(new int[] {1, 3, -1, -3, 5, 3, 6, 7}, 3),
```

JAVA (continued 9/9)

```
                new int[] {3, 3, 5, 5, 6, 7}), "window maximum");
        check(Arrays.equals(windowMaximum(new int[] {4}, 1), new int[] {4}), "single window");

        IntArrayDeque deque = new IntArrayDeque();
        int initialCapacity = deque.capacity();
        deque.addLast(2);
        deque.addFirst(1);
        deque.addLast(3);
        check(Arrays.equals(deque.logicalOrder(), new int[] {1, 2, 3}), "deque order");
        check(deque.capacity() > initialCapacity, "deque resizes");
        check(deque.removeFirst() == 1 && deque.removeLast() == 3 && deque.peekFirst() == 2,
                "both ends");
        check(deque.removeFirst() == 2 && deque.isEmpty(), "empty after removal");
        expectFailure(deque::removeFirst, NoSuchElementException.class);
        check(circularDequeMatchesArrayDeque(), "deque differential test");

        check(largestRectangleArea(new int[] {2, 1, 5, 6, 2, 3}) == 10L,
                "largest rectangle");
        check(largestRectangleArea(new int[0]) == 0L, "empty histogram");
        check(largestRectangleArea(new int[] {Integer.MAX_VALUE, Integer.MAX_VALUE})
                == 2L * Integer.MAX_VALUE, "histogram uses long area");
        expectFailure(() -> largestRectangleArea(new int[] {1, -1}),
                IllegalArgumentException.class);

        check(evaluatePostfix(new String[] {"2", "1", "+", "3", "*"}) == 9L,
                "postfix expression");
        check(evaluatePostfix(new String[] {"7", "-3", "/"}) == -2L,
                "division truncates toward zero");
        expectFailure(() -> evaluatePostfix(new String[] {"+"}),
                IllegalArgumentException.class);
        System.out.println("PASS 16 ordering-structure checks");
    }
}
```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: stack and queue simulations
- Interview Core: parentheses, next greater, and window maximum
- SDE-2 Follow-up: bounded queues and API contracts
- Challenge: histogram and monotonic-deque variants

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: DSA 10 - Linked Lists: Pointer Reasoning and Mutation

Series index

Next book: DSA 12 - Binary Search: Bounds, Answers, and Invariants

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	**JAVA 01**	Java Foundations for Problem Solving
Java	**JAVA 02**	Git and GitHub for Java Engineers
Java	**JAVA 03**	Maven and Gradle for Java Engineers
Java	**JAVA 04**	Language, OOP, and Modern Java
Java	**JAVA 05**	Collections, Streams, and I/O
Java	**JAVA 06**	JVM and Java Execution
Java	**JAVA 07**	Java Concurrency and the Memory Model
Java	**JAVA 08**	Performance, Diagnostics, and GC Incidents
Java	**JAVA 09**	Java Question Bank, Study Plan, and Reference
DSA	**DSA 01**	Time and Space Complexity for Java Interviews
DSA	**DSA 02**	Number Systems and Math Foundations for DSA Interviews
DSA	**DSA 03**	Number Systems Interview Patterns and Rapid Revision
DSA	**DSA 04**	Bit Manipulation in Java
DSA	**DSA 05**	Loop Mastery, Patterns, and Index Calculations
DSA	**DSA 06**	Arrays and Array Problem-Solving Patterns
DSA	**DSA 07**	Strings and String Problem-Solving Patterns
DSA	**DSA 08**	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	**DSA 09**	Recursion and Backtracking in Java
DSA	**DSA 10**	Linked Lists: Pointer Reasoning and Mutation
DSA	**DSA 11**	Stacks, Queues, Deques, and Monotonic Patterns
DSA	**DSA 12**	Binary Search: Bounds, Answers, and Invariants
DSA	**DSA 13**	Trees, Binary Search Trees, and Tries
DSA	**DSA 14**	Heaps, Priority Queues, Selection, and Top-K
DSA	**DSA 15**	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	**DSA 16**	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	**DSA 17**	Dynamic Programming: State, Transitions, and Optimization
Frameworks	**FW 01**	MySQL for Java Backend Interviews
Frameworks	**FW 02**	Hibernate and JPA for Java Backend Interviews
Frameworks	**FW 03**	Spring Framework for Java Engineers
Frameworks	**FW 04**	Spring Boot for Java Backend Engineers
Frameworks	**FW 05**	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	**FW 06**	Spring Data for Java Backend Engineers
Frameworks	**FW 07**	MongoDB for Java Backend Interviews
Frameworks	**FW 08**	Redis for Java Backend Interviews
Frameworks	**FW 09**	Persistence, SQL, JPA, and Caching
Frameworks	**FW 10**	Apache Kafka and Spring Kafka
Frameworks	**FW 11**	Spring Ecosystem Extensions
Frameworks	**FW 12**	Spring AI for Java Engineers
System Design	**SD 01**	Design, Backend, Testing, and Security
System Design	**SD 02**	Distributed Systems and System Design
System Design	**SD 03**	Generative AI System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 11 of 17 - Study Step DSA-11. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.